Chapter 05

Why Enums?

❌ Using String

private String status;

Problem:

user.setStatus("ACTVE");

Compiler **cannot** detect the mistake.

✅ Using Enum

private AccountStatus status;

user.setStatus(AccountStatus.ACTIVE);

Compiler only allows valid values.

What is an Enum?

An **Enum** is a Java type that represents a **fixed set of predefined constants**.

Example:

Traffic Light

▼

RED → YELLOW → GREEN

Only these values are allowed. We can't add blue, because there no blue in the enum.

Enums Created

Enum	Purpose	Values
AccountStatus	User account status	ACTIVE, INACTIVE, LOCKED, SUSPENDED
OrganizationStatus	Organization status	ACTIVE, INACTIVE, SUSPENDED
SystemRole	Standard system roles	SYSTEM_ADMIN, ORG_ADMIN, PROCESS_ANALYST, MANAGER, EMPLOYEE

Enum Usage Flow

💻 **Application** ➡️ 📌 **AccountStatus.ACTIVE** ➡️ 📦 **Java Enum** ➡️ 🧩 **Hibernate** ➡️ 🗄️ **Database**

Best Practice	Reason
Use Enum instead of String	Prevents invalid values
Use UPPER_CASE names	Java standard
Use @Enumerated(EnumType.STRING)	Stores readable values in DB
Keep enums focused	One enum for one concept
Don't use ORDINAL	Avoid data corruption

Quick Interview Notes

Question	Answer
What is an Enum?	A Java type representing a fixed set of constants.
Why use Enums?	Type safety, compiler validation, readability, maintainability.
Why not String?	Strings allow invalid values and typing mistakes.
Why EnumType.STRING?	Stores readable names and avoids issues when enum order changes.
Why avoid ORDINAL?	Reordering enum values changes stored numbers, leading to incorrect data.